Trường THPT Chuyên Lê Hồng Phong

Câu lạc bộ Tin học Trường THPT Chuyên Lê Hồng Phong
(Le Hong Phong Informatics Club)

Lý thuyết số

Gen 1.0

Năm học 2025-2026

Mục lục

1 Giới thiệu **2**

2 Một số khái niệm toán học **2**

3 Tính chia hết **3**
3.1 Bảng nhân 3
3.2 Ước chung lớn nhất 4
3.3 Bội chung nhỏ nhất 4
3.4 Thuật toán Euclid 4

4 Số nguyên tố **5**
4.1 Kiểm tra số nguyên tố 5
4.2 Sàng nguyên tố Eratosthenes 6
4.3 Phân tích thừa số nguyên tố 7

5 Hàm phi Euler **8**
5.1 Số nguyên tố cùng nhau 8
5.2 Số lượng số nguyên tố cùng nhau 8

6 Đồng dư **10**
6.1 Định nghĩa, kí hiệu và một số tính chất đặc trưng 10
6.2 Các phép toán trong đồng dư 11
6.3 Nghịch đảo modulo 11
6.4 Lũy thừa nhị phân 12

1 Giới thiệu

Các bài toán liên quan đến toán học (số học, hình học, tổ hợp, $\cdots$) thường xuyên xuất hiện trong các cuộc thi lập trình thi đấu, đặc biệt là các bài toán số học. Vì vậy, việc nắm vững những kiến thức liên quan có thể giúp ta giải quyết các bài toán đó, đồng thời ước lượng được độ phức tạp của một số thuật toán. Bài viết dưới đây sẽ cung cấp một số thông tin cơ bản về số học trong lập trình thi đấu.

2 Một số khái niệm toán học

Phép lấy tổng

Phép lấy tổng (còn gọi là sigma) được dùng để kí hiệu tổng của nhiều số theo một quy luật. Dạng tổng quát:

$$\sum_{i=a}^{b} f(i).$$

Nghĩa là ta sẽ cộng tất cả các giá trị $f(i)$ khi i chạy từ a đến b. Biểu thức trên cũng có thể viết thành $f(a) + f(a+1) + \cdots + f(b)$.

Phép lấy tích

Phép lấy tích (còn gọi là pi) được dùng để kí hiệu tích của nhiều số theo một quy luật. Dạng tổng quát:

$$\prod_{i=a}^{b} f(i).$$

Nghĩa là ta sẽ nhân tất cả các giá trị $f(i)$ khi i chạy từ a đến b. Biểu thức trên cũng có thể viết thành $f(a) \times f(a+1) \times \cdots \times f(b)$.

Phép làm tròn

Phép làm tròn được chia làm hai loại:

- Floor: $\lfloor x \rfloor$ được gọi là phép làm tròn xuống, hay số nguyên lớn nhất nhỏ hơn x.
- Ceil: $\lceil x \rceil$ được gọi là phép làm tròn lên, hay số nguyên nhỏ nhất lớn hơn x.

Hàm logarit

Trong toán học, hàm logarit (kí hiệu log) là hàm ngược của hàm mũ. Nói cách khác, nếu $a > 0$, $a \neq 1$ và $x > 0$ thì

$$\log_a x = y \quad \Longleftrightarrow \quad a^y = x.$$

Các dạng thường gặp:

- $\log_{10} x$ (còn viết là $\lg x$): logarit cơ số 10.
- $\log_2 x$: logarit cơ số 2 (thường dùng trong lập trình thi đấu).
- $\ln x$: logarit tự nhiên, tức $\log_e x$ với $e \approx 2.71828$.

Lưu ý: trong bài viết này (và trong nhiều tài liệu khác), độ phức tạp thuật toán $\log_2 x$ thường được viết gọn thành $\log x$.

Một số kí hiệu khác

Một số kí hiệu khác có thể kể đến như:

- $\forall$: với mọi
- $\exists$: tồn tại
- $\in$: thuộc (là phần tử của)
- $\cap$: giao (giao hai tập hợp)
- $\cup$: hợp (hợp hai tập hợp)
- $\mathbb{N}$: tập các số tự nhiên
- $\mathbb{Z}$: tập các số nguyên
- $\mathbb{Q}$: tập các số hữu tỉ
- $\mathbb{R}$: tập các số thực

3 Tính chia hết

3.1 Bảng nhân

Để tìm hiểu về chia hết, ta cần đi qua một khái niệm cơ bản là bảng nhân, hay thân thuộc hơn là bảng cửu chương. Hãy xem qua một ví dụ về bảng nhân 9 sau đây:

$$\begin{aligned} &\vdots \\ -3 \times 9 &= -18 \\ -1 \times 9 &= -9 \\ 0 \times 9 &= 0 \\ 1 \times 9 &= 9 \\ 2 \times 9 &= 18 \\ 3 \times 9 &= 27 \\ 4 \times 9 &= 36 \\ 5 \times 9 &= 45 \\ &\vdots \end{aligned}$$

Ta có thể quan sát thấy tập hợp các số sau:

$$\mathcal{M} = \{\ldots, -18, -9, 0, 9, 18, 27, 36, 45 \ldots\}$$

Những số này được gọi là bội của 9, hay ta có thể nói là chúng chia hết cho 9. Tổng quát hóa, ta có định nghĩa như sau:

Định nghĩa 3.1 *Nếu n xuất hiện trong bảng nhân m, hay tồn tại số nguyên k sao cho $mk = n$, thì ta có thể nói n là **bội** của m, còn m là **ước** của n.*

Trong toán học và tin học, ta sử dụng kí hiệu $|$ để biểu diễn tính chia hết:

- $m \mid n$: n chia hết cho m.
- $m \nmid n$: n không chia hết cho m.

Ví dụ, ta có $2 \mid 4$, $3 \mid 6$, $4 \mid 0$ và $6 \nmid 7$.

3.2 Ước chung lớn nhất

Với hai số nguyên không đồng thời bằng 0, **ước chung lớn nhất** của a và b (kí hiệu: $\gcd(a, b)$) là số nguyên dương lớn nhất là ước của cả a và b.

Ví dụ:

- $\gcd(12, 18) = 6$ vì các ước chung của 12 và 18 là $1, 2, 3, 6$.
- $\gcd(17, 25) = 1$ vì 17 và 25 **nguyên tố cùng nhau**.
- $\gcd(0, 15) = 15$; tổng quát $\gcd(0, x) = |x|$.
- $\gcd(-12, 18) = 6$ (thường lấy giá trị dương).

Nếu như ta muốn lấy gcd của nhiều số, ta có thể lấy gcd của từng cặp số theo bất kì thứ tự nào. Ví dụ:

$$\gcd(a_1, a_2, a_3, a_4) = \gcd(a_1, \gcd(a_2, \gcd(a_3, a_4)))$$

Trong C++, ta có thể dùng hàm `__gcd(a,b)` trong thư viện `algorithm` để tìm ước chung lớn nhất.

3.3 Bội chung nhỏ nhất

Với $a, b \neq 0$, **bội chung nhỏ nhất** (kí hiệu: $\operatorname{lcm}(a, b)$) là số nguyên dương nhỏ nhất là bội của cả a và b.

Ví dụ:

- $\operatorname{lcm}(6, 8) = 24$.
- $\operatorname{lcm}(4, 10) = 20$.
- $\operatorname{lcm}(1, n) = n$.

Ta có liên hệ sau giữa gcd và lcm. Với $a, b \neq 0$:

$$\gcd(a, b) \cdot \operatorname{lcm}(a, b) = |ab|.$$

Với $a = 12$, $b = 18$: $\gcd(12, 18) = 6$, do đó

$$\operatorname{lcm}(12, 18) = \frac{|12 \cdot 18|}{6} = 36.$$

Cài đặt

```cpp
long long lcm(long long a, long long b){
    //  Ta cần lưu ý tránh tràn số bằng cách chia trước rồi nhân sau.
    return a / gcd(a, b) * b;
}
```

3.4 Thuật toán Euclid

Thuật toán Euclid là cách nhanh nhất để tính $\gcd(a, b)$ dựa trên tính chất:

$$\gcd(a, b) = \begin{cases} |a|, & b = 0, \\ \gcd(b, a \bmod b), & b \neq 0. \end{cases}$$

Vì vậy, ta hoàn toàn có thể áp dụng tính chất trên liên tục cho đến khi $b = 0$. Khi ấy, đáp án của ta sẽ là $|a|$. Ta có thể cài đặt thuật toán Euclid bằng đệ quy cực kì đơn giản:

```
int gcd(int a, int b){
    if(b == 0) return abs(a);
    return gcd(b, a % b);
}
```

Độ phức tạp thời gian: $\mathcal{O}(\log \min(a, b))$.

4 Số nguyên tố

Một số tự nhiên n được gọi là số nguyên tố khi và chỉ khi n có đúng 2 ước dương là 1 và chính nó. Một số số nguyên tố đầu tiên là $2, 3, 5, 7, 11, 13, \cdots$.

Nếu như một số tự nhiên n không phải là số nguyên tố và $n \neq 1$, thì n được gọi là hợp số.

Gọi $\pi(n)$ là số lượng số nguyên tố nhỏ hơn hoặc bằng n. Theo định lý số nguyên tố, ta có xấp xỉ $\pi(n) \approx \frac{n}{\ln n}$.

4.1 Kiểm tra số nguyên tố

Ta có thể chia các thuật toán kiểm tra số nguyên tố thành hai nhóm lớn:

- Deterministic test: thuật toán luôn cho kết quả đúng. Một số ví dụ có thể kể đến như chia thử, kiểm tra Lucas–Lehmer.
- Probabilistic test: thường nhanh hơn các thuật toán thuộc nhóm trên, nhưng sẽ có xác suất (thường rất nhỏ) xác định hợp số là số nguyên tố. Một ví dụ tiêu biểu là thuật toán Miller–Rabin.

Trong bài viết này, ta sẽ làm quen với thuật toán **chia thử**. Thuật toán dựa trên ý tưởng duyệt qua các số từ nhỏ đến lớn để kiểm tra tính chia hết.

```
bool isPrime(int n) {
    if(n < 2) return false;
    for(int i = 2; i < n; i++)
        if(n % i == 0) return false;
    return true;
}
```

Cách này khá ổn, nhưng liệu ta có cần phải kiểm tra chia thử mọi số? Ta nhận thấy rằng chỉ cần duyệt các ước tới $\sqrt{n}$ là đủ, do ứng với mọi ước d của n, thì ta sẽ có tạo thành hai ước là d và $\frac{n}{d}$. Suy ra, nếu n không chia hết cho các số không vượt qua $\sqrt{n}$ thì cũng sẽ không chia hết cho các số lớn hơn $\sqrt{n}$.

```
bool isPrime(int n) {
    if(n < 2) return false;
    for(int i = 2; i * i <= n; i++)
        if(n % i == 0) return false;
    return true;
}
```

Độ phức tạp thời gian: $\mathcal{O}(\sqrt{n})$.

Lưu ý:

- Trong đoạn code trên, thay vì sử dụng hàm `sqrt()`, ta dùng điều kiện `i * i <= n` vì hàm lấy căn thường chậm hơn phép nhân số nguyên.
- Nếu vẫn muốn sử dụng hàm `sqrt()`, ta nên tránh việc phải tính lại `sqrt()` ở mỗi lần lặp:

```
int nSqrt = sqrt(n);
for(int i = 2; i <= nSqrt; i++)
```

4.2 Sàng nguyên tố Eratosthenes

Khi cần tìm ra toàn bộ các số nguyên tố từ 1 đến n, ta có thể duyệt từng số và kiểm tra tính nguyên tố của nó. Ý tưởng đó cho ta thuật toán $\mathcal{O}(n\sqrt{n})$.

Tuy nhiên, một nhà toán học Hy Lạp cổ tên là Eratosthenes đã phát minh ra một thuật toán hiệu quả hơn. Ban đầu, Eratosthenes đã lấy lá cọ và ghi tất cả các số từ 1 đến 100. Sau khi chọc thủng lá cọ số 1, ông chọc thủng các hợp số và giữ nguyên các số nguyên tố. Bảng số còn lại trông rất giống một cái sàng. Cho đến ngày nay, thuật toán này được phổ biến rộng rãi với tên gọi sàng nguyên tố Eratosthenes.

Quá trình

Ta có quan sát như sau: với mọi số nguyên dương $x > 1$, các bội của nó (tức kx) sẽ là hợp số nếu $k > 1$. Nhờ quan sát này, số lượng số ta phải kiểm tra sẽ giảm đi đáng kể.

Nếu ta duyệt các số từ nhỏ đến lớn, rồi với mỗi số, đánh dấu các bội của nó là hợp số thì các số không bị đánh dấu sau khi kết thúc quá trình sẽ là số nguyên tố.

```
vector<int> primes;
bool isPrime[N];

void sieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for(int i = 2; i <= n; i++) isPrime[i] = true;
    for(int i = 2; i * i <= n; i++) {
        if(isPrime[i]) {
            // Vì các số 2i, 3i, ... đã được duyệt nên ta trực tiếp duyệt từ i * i
            for(int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }
    for(int i = 2; i <= n; i++)
        if(isPrime[i]) primes.push_back(i);
}
```

Độ phức tạp thời gian: $\mathcal{O}(n \log \log n)$.

Sàng nguyên tố Eratosthenes trên đoạn

Đôi khi ta cần tìm tất cả số nguyên tố trong đoạn $[L, R]$ với độ dài tương đối nhỏ ($R - L + 1 \approx 10^7$), trong khi R có thể rất lớn (cỡ 10^{18}).

Để giải bài toán trên, ta sử dụng ý tưởng của sàng Eratosthenes kết hợp với việc tịnh tiến chỉ số mảng: đánh dấu vị trí 0 đại diện cho L, vị trí 1 cho $L + 1$, $\cdots$, và vị trí $R - L$ cho R.

```
vector<int> primes;
bool isPrime[N];

void segmentedSieve(long long L, long long R) {
    for(long long i = L; i <= R; i++) isPrime[i - L] = true;
```

```
    for(long long i = 2; i * i <= R; i++) {
        // Công thức ở dưới tìm bội đầu tiên của i nằm trong đoạn [L, R]
        // nhưng đồng thời không nhỏ hơn i * i
        for(long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i) {
            isPrime[j - L] = false;
        }
    }
    if(L == 1) isPrime[0] = false;
    for(long long i = L; i <= R; i++)
        if(isPrime[i - L]) primes.push_back(i);
}

```

Độ phức tạp thời gian: $\mathcal{O}((R - L + 1)\log R)$. Tuy nhiên, thuật toán trên chạy rất nhanh trong thực tiễn.

4.3 Phân tích thừa số nguyên tố

Số nguyên dương p được gọi là thừa số của n khi n chia hết cho p và mọi số nguyên dương n đều có thể được phân tích thành các thừa số nguyên tố:

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

trong đó có $p_1 < p_2 < \cdots < p_k$ là các số nguyên tố và $a_1, a_2, \cdots, a_k$ là các số nguyên dương.

Một số ứng dụng của thừa số nguyên tố

Nếu $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ thì ta có thể tính được:

- Số lượng ước của n là $\tau(n) = \prod_{i=1}^{k} a_i + 1$.
- Tổng các ước của n là $\sigma(n) = \prod_{i=1}^{k} (1 + p_i + \cdots + p_i^{a_i}) = \prod_{i=1}^{k} \frac{p_i^{a_i+1} - 1}{p_i - 1}$.
- Tích các ước của n là $\mu(n) = n^{\tau(n)/2}$.

Thuật toán ngây thơ

Thuật toán đơn giản nhất để phân tích thừa số nguyên tố của n là duyệt p từ 2 đến $\sqrt{n}$.

```
vector<pair<int, int>> breakdown(int n) {
    vector<pair<int, int>> result;
    for(int p = 2; p * p <= n; p++) {
        if(n % p == 0) {
            int a = 0;
            while(n % p == 0) {
                a++;
                n /= p;
            }
            result.emplace_back(p, a);
        }
    }
    if(n != 1) result.emplace_back(n, 1);
    return result;
}
```

Độ phức tạp thời gian: $\mathcal{O}(\sqrt{n})$.

Thuật toán áp dụng sàng nguyên tố

Để tăng tốc quá trình phân tích, ta tính trước mảng `minPrime`$[i]$, trong đó `minPrime`$[i]$ là ước nguyên tố nhỏ nhất của i. Sau đó, khi phân tích thừa số nguyên tố của n, thay vì kiếm p bằng cách duyệt tuần tự các số nhỏ hơn $\sqrt{n}$, ta có thể lấy trực tiếp $p =$ `minPrime`$[n]$, rồi tiếp tục thao tác như trên để thu được các thừa số nguyên tố.

```cpp
int minPrime[N];

for(int i = 2; i * i <= n; i++) {
    if(!minPrime[i]) {
        for(int j = i * i; j <= n; j += i) {
            if(!minPrime[j]) minPrime[j] = i;
        }
    }
}
for(int i = 2; i <= n; i++) if(!minPrime[i]) minPrime[i] = i;

vector<pair<int, int>> breakdown(int n) {
    vector<pair<int, int>> result;
    while(n > 1) {
        int p = minPrime[n], a = 0;
        while(n % p == 0) {
            a++;
            n /= p;
        }
        result.emplace_back(p, a);
    }
    return result;
}
```

Độ phức tạp thời gian:

- Tiền xử lý: $\mathcal{O}(n \log \log n)$.
- Phân tích thừa số nguyên tố $\mathcal{O}(\log n)$.

5 Hàm phi Euler

5.1 Số nguyên tố cùng nhau

Cặp số nguyên (x, y) được gọi là số nguyên tố cùng nhau khi và chỉ khi $\texttt{gcd}(x, y) = 1$.

- Ví dụ 1: Các cặp $(5, 3), (2, 7), (10, 9)$ là số nguyên tố cùng nhau.
- Ví dụ 2: Các cặp $(4, 2), (9, 12), (25, 10)$ không phải số nguyên tố cùng nhau do ước chung lớn nhất lớn hơn 1.

5.2 Số lượng số nguyên tố cùng nhau

Hàm phi Euler $\varphi(n)$ cho biết số lượng số nguyên tố cùng nhau với n trong phạm vi bé hơn hoặc bằng n.

- Ví dụ: các số nguyên tố cùng nhau với 10 và bé hơn hoặc bằng 10 là: $1, 3, 7, 9$. Do có 4 số như thế nên $\varphi(10) = 4$.

- Tương tự, ta có $\varphi(n)$ của 10 số nguyên đầu tiên là:

$$\begin{aligned}
\varphi(1) &= 1 \\
\varphi(2) &= 1 \\
\varphi(3) &= 2 \\
\varphi(4) &= 2 \\
\varphi(5) &= 4 \\
\varphi(6) &= 2 \\
\varphi(7) &= 6 \\
\varphi(8) &= 4 \\
\varphi(9) &= 6 \\
\varphi(10) &= 4.
\end{aligned}$$

Trường hợp tổng quát

Đối với một số nguyên dương n bất kì, ta có công thức tổng quát để tính phi như sau:

$$\varphi(n) = n \times \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

trong đó p là một trong các thừa số nguyên tố của n.

- Ví dụ 1 - Tính $\varphi(10)$: do $10 = 2 \cdot 5$ nên $\varphi(10) = 10\left(1 - \frac{1}{2}\right)\left(1 - \frac{1}{5}\right) = 4$.
- Ví dụ 2 - Tính $\varphi(25)$: do $25 = 5^2$ nên $\varphi(25) = 25\left(1 - \frac{1}{5}\right) = 20$.

Trường hợp chỉ có một thừa số nguyên tố

Nếu biết trước n là lũy thừa bậc k của một số nguyên tố p ($n = p^k$) thì ta có công thức tính φ khác như sau:

$$\varphi(n) = p^k - p^{k-1}$$

- Ví dụ: $\varphi(25) = 5^2 - 5^1 = 20,\ \varphi(32) = 2^5 - 2^4 = 16$.
- Phần chứng minh: các số nguyên tố không cùng nhau với p^k tương ứng các bội số của p: $p, 2p, 3p, \ldots, p^k$. Do có p^{k-1} số như thế nên các số nguyên tố cùng nhau với p^k chính là $p^k - p^{k-1} = \varphi(p^k)$. Ngoài ra, bạn có thể dùng cơ sở này để tự chứng minh công thức tổng quát ở phía trên.

Trường hợp là tích 2 số nguyên

Để tính phi của tích 2 số nguyên, ta cũng có công thức khác là:

$$\varphi(a \cdot b) = d \times \frac{\varphi(a) \cdot \varphi(b)}{\varphi(d)}$$

với d là ước chung lớn nhất của a và b.

Code mẫu

Dưới đây là code mẫu tính $\varphi(n)$ với n là 1 số nguyên dương cho trước:

```cpp
int phi(int n){
    int res = n;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0){
            while (n % i == 0){
                n /= i;
            }
            res -=  res / i;
        }
    if (n > 1)
        res -= res / n;
    return res;
}
```

Độ phức tạp thời gian: $\mathcal{O}(\sqrt{n})$.

Ta có thể chỉnh sửa thuật toán trên để tính $\varphi(i)$ cho mọi số nguyên bé hơn hoặc bằng n, thông qua thuật toán sàng nguyên tố:

```cpp
int phi[N];

void sievePhi(int n){
    for(int i = 1; i <= n; ++i) phi[i] = i;
    for(int i = 2; i <= n; ++i){
        if(phi[i] == i){ // i là số nguyên tố
            for(int j = i; j <= n; j += i){
                phi[j] -= phi[j] / i;  // tương đương phi[j] * (1 - 1/i)
            }
        }
    }
}
```

Độ phức tạp thời gian: $\mathcal{O}(n \log \log n)$

6 Đồng dư

6.1 Định nghĩa, kí hiệu và một số tính chất đặc trưng

Cho $m \in \mathbb{N}$, $m \geq 2$. Ta nói a **đồng dư** b theo modulo m (kí hiệu $a \equiv b \pmod{m}$) nếu

$$m \mid (a - b).$$

Nói cách khác, a và b có cùng số dư khi chia cho m. Ví dụ, ta có:

- $17 \equiv 5 \pmod{12}$ vì $17 - 5 = 12$ chia hết cho 12.
- $-1 \equiv 6 \pmod{7}$ vì $-1 - 6 = -7$ chia hết cho 7.
- $25 \equiv 1 \pmod{8}$ vì 25 chia 8 dư 1.

Tính chất:

Phép đồng dư thoả mãn một số tính chất cơ bản sau:

- Phản xạ: $a \equiv a \pmod{m}$

- Đối xứng: Nếu $a \equiv b \pmod{m}$ thì $b \equiv a \pmod{m}$.
- Bắc cầu: Nếu $a \equiv b \pmod{m}$ và $b \equiv c \pmod{m}$ thì $a \equiv c \pmod{m}$.

Ngoài ra, ta còn có thêm một số tính chất khác:

- $a \equiv b \pmod{m} \iff a + km \equiv b + \ell m \pmod{m}$ với mọi $k, \ell \in \mathbb{Z}$.
- Nếu $a \equiv b \pmod{m}$ thì $ka \equiv kb \pmod{m}$ với mọi $k \in \mathbb{Z}$.
- Nếu $\gcd(k, m) = 1$ và $ka \equiv kb \pmod{m}$ thì $a \equiv b \pmod{m}$.

6.2 Các phép toán trong đồng dư

Trong C++, phép chia dư được viết là `a % m`. Với $m > 0$ và $a \geq 0$ thì:

$$a \bmod m = a - m \left\lfloor \frac{a}{m} \right\rfloor, \quad 0 \leq a \bmod m \leq m - 1.$$

Nếu a âm thì `a % m` có thể âm (cùng dấu với a), nên ta thường chuẩn hóa về không âm bằng cách lấy `(a % m + m) % m`. Ngoài ra, ta còn có một số phép toán cơ bản (với $m > 0$) như sau:

- $(a + b) \bmod m = \big((a \bmod m) + (b \bmod m)\big) \bmod m$.
- $(a - b) \bmod m = \big((a \bmod m) - (b \bmod m)\big) \bmod m$.
- $(a \cdot b) \bmod m = \big((a \bmod m) \cdot (b \bmod m)\big) \bmod m$.

Ta cũng cần phải lưu ý về thứ tự ưu tiên toán tử trong C++: `a + b % m` được hiểu là `a + (b % m)`. Nên ta cần thêm các dấu ngoặc ở vị trí phù hợp để đảm bảo tính đúng đắn.

6.3 Nghịch đảo modulo

Trong nhiều bài toán, ta cần "chia" trong modulo. Tuy nhiên, phép chia không phải lúc nào cũng hợp lệ; ta chỉ có thể chia cho a theo modulo m khi a có **nghịch đảo modulo**.

Định nghĩa

Cho $m \geq 2$. Số nguyên x được gọi là **nghịch đảo modulo** của a theo modulo m nếu:

$$ax \equiv 1 \pmod{m}.$$

Điều kiện tồn tại và tính duy nhất:

- a có nghịch đảo modulo m **khi và chỉ khi** $\gcd(a, m) = 1$.
- Nếu tồn tại, nghịch đảo là **duy nhất theo modulo** m, tức nếu $ax \equiv 1 \pmod{m}$ và $ay \equiv 1 \pmod{m}$ thì $x \equiv y \pmod{m}$.

Cách tìm nghịch đảo

- Định lý Fermat nhỏ: Nếu p là số nguyên tố và $a \not\equiv 0 \pmod{p}$ thì

  $$a^{p-1} \equiv 1 \pmod{p} \Rightarrow a^{p-2} \equiv a^{-1} \pmod{p}.$$

  Khi đó có thể dùng lũy thừa nhị phân để tính $a^{p-2} \bmod p$.
- Định lý Euler:

  $$a^{\phi(m)} \equiv 1 \pmod{m}$$

  Với $\phi(m)$ là phi hàm Euler của m, ta suy ra:

  $$a \times a^{\phi(m)-1} \equiv 1 \pmod{m}$$

  Vậy nghịch đảo modulo sẽ bằng $a^{-1} = a^{\phi(m)-1} \bmod m$.

6.4 Lũy thừa nhị phân

Định nghĩa

Lũy thừa nhị phân (binary exponentiation) là phương pháp tính nhanh a^n với n là số nguyên không âm, bằng cách khai thác tính chất:

$$a^b = \begin{cases} 1, & b = 0 \\ a, & b = 1 \\ a^{\frac{b}{2}} \times a^{\frac{b}{2}}, & \text{nếu } b \text{ là số chẵn} \\ a^{\lfloor \frac{b}{2} \rfloor} \times a^{\lfloor \frac{b}{2} \rfloor} \times a, & \text{nếu } b \text{ là số lẻ} \end{cases}$$

Ý tưởng là tách n theo dạng nhị phân (liên tục chia n cho 2), mỗi bước chỉ cần bình phương và nhân thêm khi n lẻ. Nếu tính trực tiếp a^n bằng cách nhân a với chính nó n lần thì cần n phép nhân. Với lũy thừa nhị phân, số phép nhân giảm xuống còn tầm $\log n$.

Ứng dụng

- Tính lũy thừa lớn nhanh, đặc biệt khi n rất lớn (cỡ 10^9 trở lên).
- Tính lũy thừa theo modulo: $a^n \bmod m$, dùng trong bài toán đồng dư và nghịch đảo modulo (nhờ định lý Fermat nhỏ khi m là số nguyên tố).
- Hỗ trợ các thuật toán cần nhân lặp lại nhiều lần, ví dụ tính ma trận lũy thừa để đếm số cách trong bài toán tổ hợp hay quy hoạch động.

Cài đặt (đệ quy)

```cpp
int multiply(int a, int b) {
    // Ép kiểu thành long long để tránh tràn số
    return 1LL * a * b % MOD;
}

int power(int a, int b){
    if(b == 0) return 1;
    if(b == 1) return a % MOD;
    int half = power(a, b / 2);
    if(b % 2 == 1) return multiply(multiply(half, half), a);
    return multiply(half, half);
}
```